



GitHub Trending Top 10

每日热门开源项目深度解读

2026-08-21

今日榜单

- 1 modular/modular** Mojo
The Modular Platform (includes MAX & Mojo) NEW
- 2 mattpocock/skills** Shell
Skills for Real Engineers. Straight from my .agents directory. 2天
- 3 AprilNEA/OpenLogi** Rust
A native, local-first alternative to Logitech Options+, written in Rust — re... NEW
- 4 obra/superpowers** Shell
An agentic skills framework & software development methodology that works. 2天
- 5 cursor/plugins** TypeScript
Cursor plugin specification and official plugins NEW
- 6 santifer/career-ops** JavaScript
Open-source AI job search: scan job portals, evaluate listings with a structured... 2天
- 7 akitaonrails/ai-memory** Rust
Solution for long term memory for agent coding CLIs and to facilitate handoff be... 4天
- 8 harry0703/MoneyPrinterTurbo** Python
利用 AI 大模型和自动化工作流，根据主题或关键词一键生成高清短视频。Generate HD short vi... 4天
- 9 agent-substrate/substrate** Go
Agent Substrate: the core system NEW
- 10 chaitanyagiri/munder-diffllin** TypeScript
local multi-agent harness 3天

#1

Mojo

NEW

modular

The Modular Platform (includes MAX & Mojo)

Stars **28,323** Forks **3,020** Today **+268**

<https://github.com/modular/modular>

项目简介：Modular Platform 是一个面向 AI 开发与部署的统一平台，包含 MAX 框架（用于模型推理与部署）和 Mojo 语言（一种面向 AI 的编程语言）。它旨在解决 AI 基础设施碎片化问题，让开发者用更少的代码和更高的性能构建、优化并运行 AI 模型。

核心功能：1) Mojo 编译器与标准库，提供 Python 语法兼容但性能接近 C/C++ 的编程体验；2) MAX 加速库（kernels），包含针对硬件优化的算子实现；3) MAX 推理服务器，提供 OpenAI 兼容的 API 端点，方便快速部署；4) MAX 模型流水线，支持 Python 构建的模型图；5) 丰富的代码示例，覆盖从入门到生产级场景。

适用场景：主要面向 AI 工程师、系统性能优化人员和 ML 基础设施团队。典型场景包括：需要低延迟推理的在线服务、在异构硬件（CPU/GPU）上运行大模型、以及希望用 Python 语法开发高性能 AI 应用的开发者。

技术亮点：Mojo 语言融合了 Python 的易用性与 Rust/C++ 的性能，其编译器基于 LLVM 技术栈，支持零开销抽象和线性类型系统。MAX 框架的加速库直接针对底层硬件优化，配合模型流水线可显著减少推理延迟。项目采用 Apache 2.0 许可证，但 MAX 部分使用社区许可，体现了开源与商业化的平衡。

上手建议：新手可从官方快速入门指南开始，先尝试用 MAX 部署一个预训练模型（如 Llama），感受其推理性能。若想贡献代码，建议先阅读贡献指南，从 Mojo 标准库或 MAX 加速库的简单模块入手，修复 bug 或补充文档是低门槛的切入点。目前 Mojo 编译器暂不接收外部贡献，专注其他模块即可。

#2

Shell

连续 2 天

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **227,916** Forks **19,512** Today **+2,192**

<https://github.com/mattpocock/skills>

项目简介：这是一个由知名 TypeScript 教育者 Matt Pocock 开源的 AI 编程智能体（Agent）技能包集合，旨在让 AI 编码助手真正完成"工程化"工作，而非简单的"vibe coding"（随性编码）。它解决的核心问题是：AI 编码代理经常误解需求、输出冗长，且缺乏对真实工程流程的掌控。

核心功能：1) `/grill-me` 与 `/grill-with-docs`——通过结构化"拷问"让 AI 在动手前充分澄清需求，消除沟通偏差；2) `/setup-matt-pocock-skills`——一次性配置项目，自动对接 GitHub/Linear 等 Issue 跟踪器及标签体系；3) `/triage`——利用标签对任务进行智能分类与优先级排序；4) **可组合的小型技能**——每个技能独立、易修改，可自由搭配，不绑定特定模型或流程。

适用场景：适用于使用 Claude Code、Codex 等 AI 编程助手的专业开发者，尤其是需要处理复杂需求、多任务管理或希望将 AI 深度整合进现有工程流程（如 Issue 驱动开发）的团队。新手也能借此建立更严谨的 AI 协作习惯。

技术亮点：项目采用"小而美"的设计哲学，摒弃了 GSD、BMAD 等重流程框架，将技能拆分为可独立修改的 Markdown 文件（SKILL.md），既保持了透明可控，又支持通过 `npx skills` 或 Claude Code 插件两种方式灵活安装。特别值得一提的是，它支持"订阅模式"（插件自动更新）与"Fork 模式"（本地文件自由修改）两种哲学，兼顾了便利性与可定制性。

上手建议：推荐先通过 `npx skills@latest add mattpocock/skills` 安装到本地仓库，并确保选中 `setup-matt-pocock-skills`。然后在代理中运行 `/setup-matt-pocock-skills` 完成一次配置，优先体验 `/grill-me` 与 `/grill-with-docs` 这两个最受欢迎的技能，感受需求对齐带来的效率提升。若想贡献，可直接 Fork 仓库并修改 `skills/` 目录下的技能文件，遵循现有的 ADR 文档规范提交。

#3

Rust

NEW

OpenLogi

A native, local-first alternative to Logitech Options+, written in Rust — remap buttons, DPI, and SmartShift over HID++. No account, no telemetry.

Stars **12,379** Forks **339** Today **+1,545**

<https://github.com/AprilNEA/OpenLogi>

项目简介：OpenLogi 是一个用 Rust 编写的罗技 Options+ 开源替代品，主打本地优先、无需账号和遥测。它通过 HID++ 和 UVC 协议解锁罗技鼠标、键盘和摄像头的完整能力，解决了官方软件不支持 Linux、臃肿且涉及隐私的问题。

核心功能：支持按键重映射、DPI 调节、SmartShift 滚轮模式切换和手势绑定；提供纯文本 TOML 配置文件和 CLI 工具，便于脚本化与跨设备同步；还包含 Litra 灯光控制、摄像头 UVC 参数调节和 F 键全局重映射等高级功能。

适用场景：适合对隐私敏感、使用 Linux 系统或厌倦官方软件臃肿体验的罗技外设用户。尤其适合需要精细控制鼠标手势、DPI 预设和摄像头画质的开发者、设计师和直播用户。

技术亮点：采用 Rust 原生实现（基于 GPUI 框架）保证轻量性能；通过 OS 输入钩子实现应用级配置自动切换；摄像头控制直接写入 UVC 硬件，无需常驻进程；配置与代码完全开放，支持多平台（macOS/Linux/Windows）一等公民体验。

上手建议：先退出 Logi Options+ 避免 HID++ 冲突，从 GitHub Releases 下载对应系统安装包或通过 Homebrew 安装。想贡献代码可先阅读 docs 目录下的多语言文档，关注项目活跃的 Telegram 频道获取开发动态，当前仍处快速迭代期，建议收藏仓库等待稳定版。

#4

Shell

连续 2 天

superpowers

An agentic skills framework & software development methodology that works.

Stars **275,299** Forks **24,623** Today **+727**

<https://github.com/obra/superpowers>

项目简介：Superpowers 是一套面向 AI 编码代理的软件开发方法论，通过可组合的技能模块和初始指令，让 AI 从“直接写代码”转变为“先理解需求、再设计方案、最后按计划执行”的工程化流程。它解决的核心问题是 AI 编码时缺乏项目上下文和工程纪律，容易产出偏离需求的代码。

核心功能：1) 自动触发的技能体系，无需手动切换模式；2) 需求规格提取与分块确认机制，避免误解；3) 基于 TDD（测试驱动开发）、YAGNI（不做多余功能）和 DRY（不重复代码）原则的实现计划生成；4) 子代理驱动的自主开发流程，可连续数小时按计划执行任务。

适用场景：适合使用 Claude Code、Cursor、Gemini CLI、GitHub Copilot 等主流 AI 编程工具的个人开发者或团队，尤其是需要 AI 独立完成较大功能模块、希望保持代码质量和项目一致性的场景。企业用户还可获得商业支持。

技术亮点：项目采用“技能即代码”的架构，将工程实践封装为可组合的 Shell 脚本技能，通过会话启动钩子自动加载，实现了跨 14+ 种 AI 编码工具的插件化分发。其核心设计理念是让 AI 先“思考”再“动手”，通过子代理分工模拟真实团队协作流程。

上手建议：如果你是用户，根据自己使用的编码工具（如 Claude Code 或 Cursor）按 README 中的安装命令一键安装即可；如果你是开发者，可先阅读仓库中的技能模块源码，了解如何将工程实践封装为可复用的技能，再通过 PR 贡献新的技能或改进现有流程。

#5

TypeScript

NEW

plugins

Cursor plugin specification and official plugins

Stars **4,228** Forks **348** Today **+449**

<https://github.com/cursor/plugins>

项目简介：这是 Cursor 官方发布的插件仓库，为 AI 编程助手提供了一套标准化的插件规范（.cursor-plugin/plugin.json）和丰富的官方插件集。它解决的是如何让 AI 编程助手更深度地集成外部工具、SaaS 服务和开发者工作流的问题，相当于 Cursor 的“应用商店”底座。

核心功能：① 插件规范与脚手架——通过 create-plugin 快速生成和校验插件；② 开发工作流增强——包括代码审查（Thermos）、PR 审阅画布、CLI 设计模式、并行任务编排（Orchestrate）等；③ 第三方 SaaS 集成——覆盖 Gmail、GitHub、Salesforce、Zoom、HubSpot 等 20+ 主流工具，让 AI 能直接操作真实业务数据；④ 团队协作套件——如 Cursor Team Kit 提供 CI、代码审查和发布流程的自动化。

适用场景：主要面向两类用户——普通开发者希望让 Cursor 直接操作 GitHub、Gmail 等日常工具，减少上下文切换；团队/平台工程师则可以利用插件规范构建内部工作流（如代码审查流水线、文档维护、自动化测试），把 AI 助手变成团队基础设施的一部分。

技术亮点：插件采用“目录即插件”的极简设计，每个插件独立成目录并附带 manifest 清单，降低了分发和管理的复杂度。值得关注的是 continual-learning 和 ralph-loop 这类探索性插件，它们尝试让 AI 通过转录驱动的记忆更新和自引用循环来持续改进自身行为，代表了 AI 编程工具从“一次性任务执行”向“自我进化”演进的方向。

上手建议：如果只是想使用插件，直接在 Cursor 的插件市场搜索安装即可，无需处理代码。如果想开发自己的插件，建议从 create-plugin 目录开始，阅读插件规范和官方示例（如 cli-for-agent 是很好的起步参考），然后参考 third_party/ 下现有集成了解如何对接外部 API。仓库结构清晰，每个插件都是独立的，贡献门槛较低。

#6

JavaScript

连续 2 天

career-ops

Open-source AI job search: scan job portals, evaluate listings with a structured A-F rubric into a 1.0-5.0 score, tailor your CV, track applications — runs locally in your AI coding CLI (Claude Code, Codex, OpenCode, Antigravity...)

Stars **67,084** Forks **12,849** Today **+816**

<https://github.com/santifer/career-ops>

项目简介：career-ops 是一个开源 AI 求职系统，旨在扭转求职中“公司用 AI 筛人、求职者被动等待”的不对称局面。它运行在本地 AI 编程 CLI（如 Claude Code）中，通过多智能体协作自动完成职位扫描、评估、简历定制和申请跟踪的全流程。

核心功能：一是自动扫描多个招聘门户，批量抓取职位列表；二是用结构化的 A-F 评分标准评估每个职位，换算为 1.0-5.0 的量化分数，帮求职者快速筛掉低质量岗位；三是根据目标职位自动定制个性化简历；四是提供申请追踪看板，管理求职进度。

适用场景：主要面向正在大规模海投的技术求职者，尤其是那些每天要浏览数百个职位、却难以判断哪些真正值得投入时间的开发者。对于同时投递多家公司、需要批量定制简历的候选人，这套工具能显著节省时间。

技术亮点：项目采用“Agent Skill Standard”标准设计，因此不绑定单一 AI CLI，可运行于 Claude Code、Codex、OpenCode、Antigravity 等多种环境。它本质上是把 AI 编程助手当成自动化执行引擎，通过提示词和工具调用驱动多智能体协作，而非传统的独立应用架构。

上手建议：如果你是使用者，从 README 中的快速开始指南入手，在你的 AI CLI 中安装该 Skill 后即可开始扫描职位。如果你想贡献代码，可以从 GitHub Issues 中寻找标注“good first issue”的任务，或参与 Discord 社区了解开发方向；项目以 JavaScript 为主，熟悉 Node.js 和 Playwright 会有帮助。

#7

Rust

连续 4 天

ai-memory

Solution for long term memory for agent coding CLIs and to facilitate handoff between different agent vendors

Stars **3,813** Forks **286** Today **+332**

<https://github.com/akitaonrails/ai-memory>

项目简介：

ai-memory 是一个为 AI 编码代理提供长期记忆的 Rust 工具。它解决的核心痛点是：当你从 Claude Code 切换到 OpenAI Codex 或其他代理时，新代理无法自动继承之前的架构决策、失败尝试和未决问题，导致重复劳动。该项目通过统一的记忆层，让不同厂商的 AI 代理在同一个工作目录下无缝接力。

核心功能：

- 跨代理记忆共享：**支持 Claude Code、Codex、Gemini CLI、Cursor 等十余种主流 AI 编码工具，通过 MCP 配置和生命周期钩子自动捕获和注入上下文。
- 会话终结与交接：**提供 `finalize-session` 命令生成总结和交接文档，确保代理切换时信息不丢失。
- 托管工作流：**`ai-memory run` 提供跨工具的透明连续性，自动恢复会话数据库并注入便携上下文。
- 多平台支持：**Linux、macOS、Windows（WSL2/原生）均有对应安装路径和兼容方案。

适用场景：

主要面向重度依赖 AI 编码代理的开发者，尤其是那些会在不同工具间切换（如 Claude Code 与 Codex 混用）、或需要长期维护大型代码库的团队。也适用于需要将 AI 辅助开发工作交接给同事或 CI/CD 流程的场景。

技术亮点：

项目采用 Rust 实现，性能开销极低；通过 MCP（Model Context Protocol）标准协议实现代理无关的集成，而非为每个工具写死适配层。其生命周期钩子设计非常精细，能区分“轮次结束”和“会话结束”，并针对不同代理的钩子差异（如 Codex 无真正的会话结束钩子）提供了优雅的降级方案。

上手建议：

从 README 中的 Support Matrix 开始，找到你当前使用的代理对应的安装方式（如 Claude Code 用 `install-mcp`，Codex 需手动运行 `finalize-session`）。先在小项目上验证记忆捕获和交接流

程，再逐步扩展到日常开发。若想贡献代码，可从 Rust 源码入手，重点关注 `src/hooks` 和 `src/mcp` 目录下的代理适配逻辑。

#8

Python

连续 4 天

MoneyPrinterTurbo

利用 AI 大模型和自动化工作流，根据主题或关键词一键生成高清短视频。Generate HD short videos from a topic or keyword with an automated AI workflow.

Stars **113,440** Forks **17,204** Today **+2,761**

<https://github.com/harry0703/MoneyPrinterTurbo>

项目简介：MoneyPrinterTurbo 是一款利用 AI 大模型和自动化工作流，将主题或关键词一键转化为高清短视频的开源工具。它解决了内容创作者在视频生产过程中脚本撰写、素材匹配、字幕生成和配音合成等环节耗时费力的问题，将整个流程压缩到分钟级。

核心功能：核心功能包括基于 LLM 的自动化脚本生成、智能匹配视频素材、自动生成字幕并合成配音，以及最终将元素合成为高清视频。此外，项目提供 WebUI 和 API 两种交互方式，方便不同技术背景的用户使用。

适用场景：适用于自媒体运营者、营销人员、教育内容创作者等需要高频产出短视频的群体。无论是制作知识科普、产品推广还是新闻资讯类视频，只要输入一个简单的主题，就能快速生成成品，极大提升内容生产效率。

技术亮点：项目技术架构巧妙地将 LLM 的“创意能力”与自动化管线的“执行能力”结合，让 AI 不仅撰写文案，还负责提炼素材搜索关键词，从而提升素材匹配的精准度。同时，项目采用模块化设计，支持对接多种 LLM 和 TTS 服务，具备良好的扩展性和灵活性。

上手建议：对于使用者，建议从项目 Release 页面下载对应平台的安装包，或按照 README 指引通过 Docker 快速部署，体验从关键词到成片的完整流程。对于想贡献代码的开发者，可以从阅读项目文档中关于 API 调用和插件扩展的部分入手，了解其工作流抽象和模块接口设计。

#9

Go

NEW

substrate

Agent Substrate: the core system

Stars **1,499** Forks **255** Today **+22**

<https://github.com/agent-substrate/substrate>

项目简介：Agent Substrate 是一个面向大规模 AI Agent 部署的高密度运行时环境，核心解决的是“Agent 太多、机器太少”的资源利用问题。它通过将大量空闲的 Agent 映射到少量物理机器上，实现 30 倍以上的超卖，同时保证亚秒级的唤醒延迟。

核心功能：一是完整的生命周期管理，支持 Agent 的创建、销毁、挂起和恢复，且恢复速度在亚秒级别；二是状态持久化，通过全量快照保存 Agent 的内存和文件系统状态，让 Agent 在休眠后能完美恢复；三是多沙箱技术支持，同时支持 microVM 和 gVisor，提供一致的运维体验；四是基于 Kubernetes 构建，复用 Pod 管理和自动扩缩容能力，实现基础设施的统一管理。

适用场景：最适合运行“长时间空闲、偶尔响应”的状态ful Agent 应用，比如编码助手（Claude Code、CodeX）、工具调用型 Agent（MCP 服务器）、以及需要大规模并行训练的强化学习场景。对于需要同时维护成千上万个 Agent 实例、但又不想为每个 Agent 单独分配一台机器的团队尤其有价值。

技术亮点：其核心洞察在于“Agent 大多数时间都是空闲的”这一观察，据此设计了 Actor-Worker 映射模型，将大量逻辑上的 Agent（Actor）动态调度到少量物理资源（Worker）上。另一个亮点是框架无关性——它在内核层面管理标准 OCI 容器，因此无论上层用 LangChain、ADK 还是任何自定义框架，都能无缝接入。

上手建议：项目目前处于早期开发阶段，API 不稳定，不建议直接用于生产环境。想要尝试的话，可以从 README 中的 Counter Demo 入手，按照文档在 Kubernetes 集群中复现“250 个 Actor 跑在 8 个 Pod 上”的演示；想要贡献代码，建议先加入 ate-dev Google Group 了解开发动态，再围绕生命周期管理、调度策略或沙箱集成等方向寻找切入点。

#10

TypeScript

连续 3 天

munder-difflin

local multi-agent harness

Stars **3,262** Forks **360** Today **+507**<https://github.com/chaitanyagiri/munder-difflin>

项目简介：

Munder Difflin 是一个开源的多智能体编排工具（multi-agent harness），目标是把用户已有的终端 AI 编程助手（如 Claude Code、Codex、Grok 等）变成一个个“数字分身”，在本地自动协作完成复杂任务。它解决的核心问题是：单个 AI 助手能力有限，而手动协调多个助手又太费时，这个项目让多个助手像办公室员工一样自动分工、沟通和记忆。

核心功能：

- 多引擎接入：**支持 Claude Code、Gemini、Codex、Grok、Kimi、Qwen 等主流终端 AI CLI，自带 API Key 或本地模型均可。
- 智能编排：**通过“GOD agent”（Michael）作为主管，自动路由任务、分配工作、协调多个智能体之间的消息传递。
- 持久记忆：**内置 Markdown 优先的记忆层，带语义检索索引，让智能体跨会话记住上下文，毫秒级召回。
- 可视化办公桌：**用 Pixi.js 渲染 2D 办公室场景，智能体以虚拟形象走动、收发消息，直观展示工作状态。

适用场景：

适合重度使用 AI 编程助手、需要并行处理多个编程任务的开发者或团队。典型场景包括：你离开电脑时让多个智能体继续处理不同模块的代码任务、自动化执行多步骤开发流程（如重构+测试+文档）、或在一个复杂代码库中让多个智能体分别负责不同子系统并协作。任何想把自己的 AI 工作流“工业化”的人都会感兴趣。

技术亮点：

架构上采用“包装真实 CLI 进程”而非重新实现 AI 逻辑，通过 node-pty 保持字节级真实的终端交互，兼容性极强。记忆层采用“Markdown 文件 + 语义索引”的轻量方案，既保证可读性又实现快速检索，避免了传统向量数据库的复杂部署。整个系统是事件驱动的消息路由架构，智能体之间通过邮箱解耦，扩展新引擎只需实现一个适配器。

上手建议：

如果你是使用者：先安装依赖的 AI CLI（如 Claude Code），然后克隆项目、`npm install` 启动桌面应用，按文档配置好 API Key 即可体验。如果你是贡献者：从项目的 CONTRIBUTING.md 开始，了解架构文档和设计系统，优先看 router 和 memory 模块——它们是核心逻辑所在。项目目前是“working prototype”阶段，PR 欢迎，加入 Discord 可以获取实时开发动态。

数据来源: github.com/trending

报告日期: 2026-08-21

由 DeepSeek AI 提供智能分析 | 自动生成